

Data Types in SQL Server (Transact-SQL)



Presenter: Prachi Kabra

Date: 2026.4.3





Contents

- 01** Section 1: Foundation & Importance
- 02** Section 2: Core Data Type Categories
- 03** Section 3: Advanced Concepts & Practical Applications

The background is a dark blue gradient with intricate white patterns. These patterns include concentric circles, wavy lines, and a grid of small dots, creating a sense of depth and movement. The overall aesthetic is modern and technological.

01

Section 1: Foundation & Importance

What Are Data Types?

Core Concept

Data types define the kind of data a column can store and how SQL Server allocates memory and processes that data.

Memory & Validation

Data types determine memory allocation, enable database-level validation, ensure consistency, and prevent invalid data entry.



Define Allowed Values

Data types define allowed values and operations for database columns, enabling type-safe query execution.

Query Optimization

Data types support type-safe query execution and optimization, improving database performance and reliability.

Why Data Types Matter

01

Data Integrity

Enforce valid data entry and prevent corrupt or inconsistent records in databases.

02

Performance Optimization

Correct types reduce query execution time and minimize resource consumption efficiently.

03

Storage Efficiency

Appropriate types minimize disk space usage and reduce overall memory requirements.

04

Query Optimization

Enable SQL Server query optimizer to create better execution plans automatically.

05

Application Reliability

Type safety reduces runtime errors and prevents unexpected application behavior.

06

Compliance & Security

Support regulatory requirements and enforce comprehensive data validation rules.

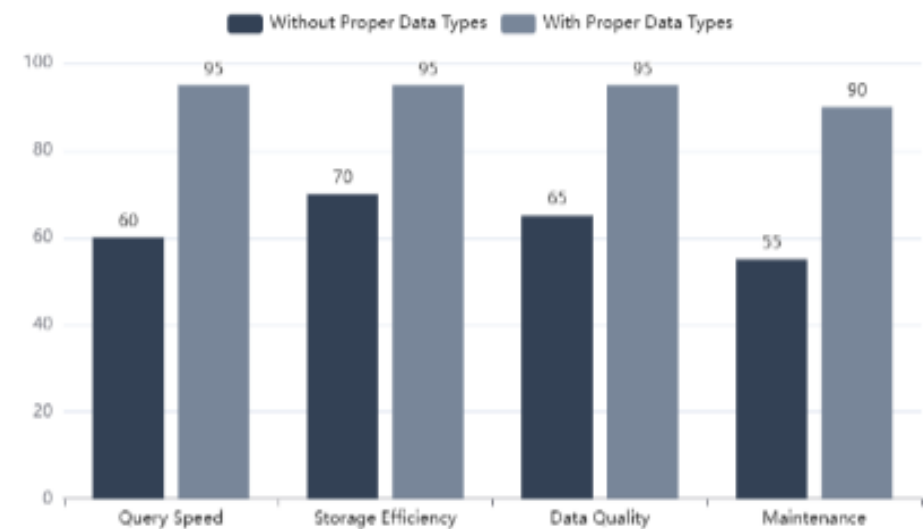
Business Impact Overview

Impact Metrics Analysis

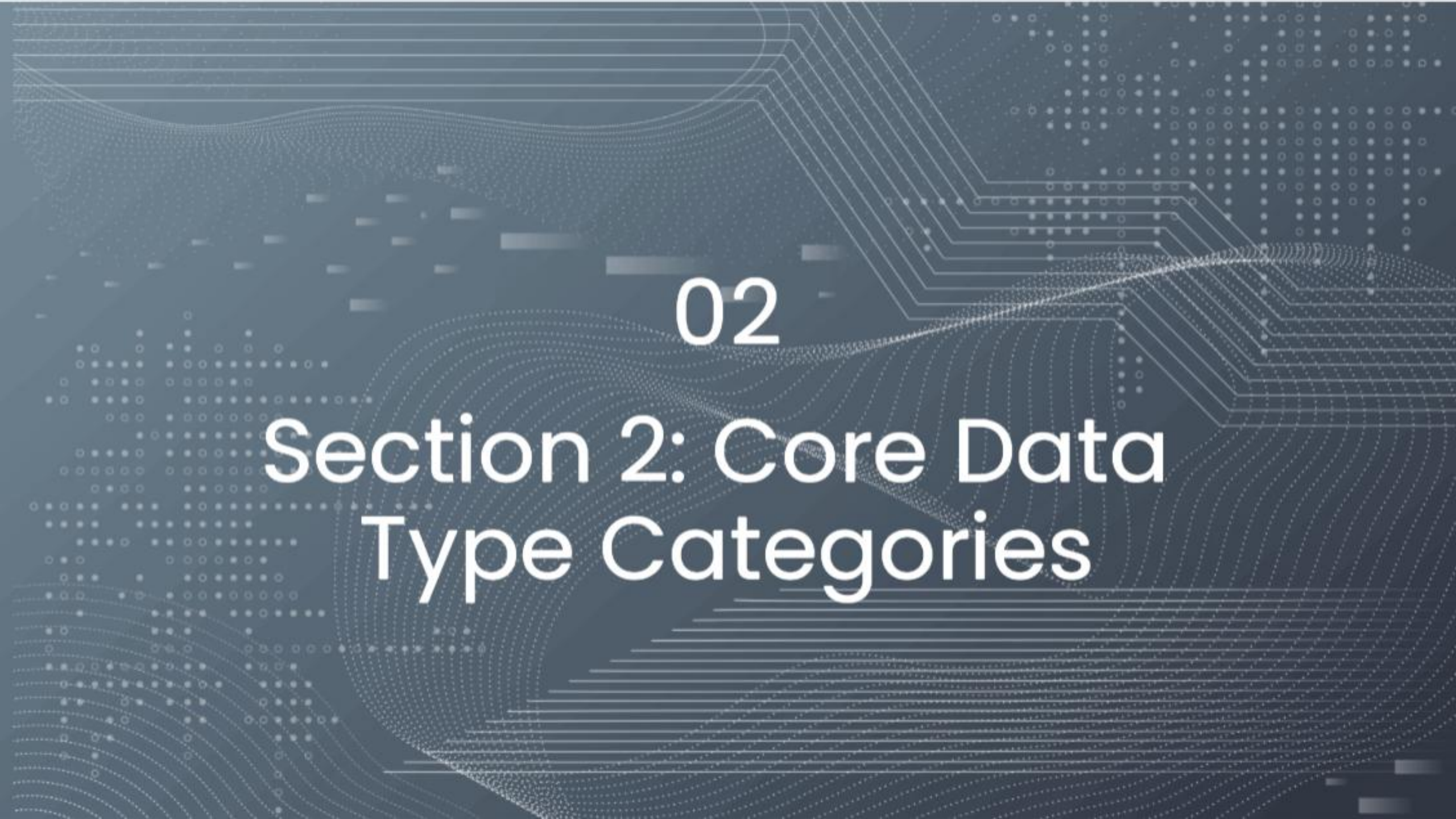
Proper data types optimize query speed, reduce storage costs, enforce data quality, and simplify maintenance compared to improper type implementation approaches.

Real-World Business Scenario

E-commerce platform with 1M+ transactions experienced 40% slower queries and 30% extra storage costs due to incorrect data type assignments in database schema design.



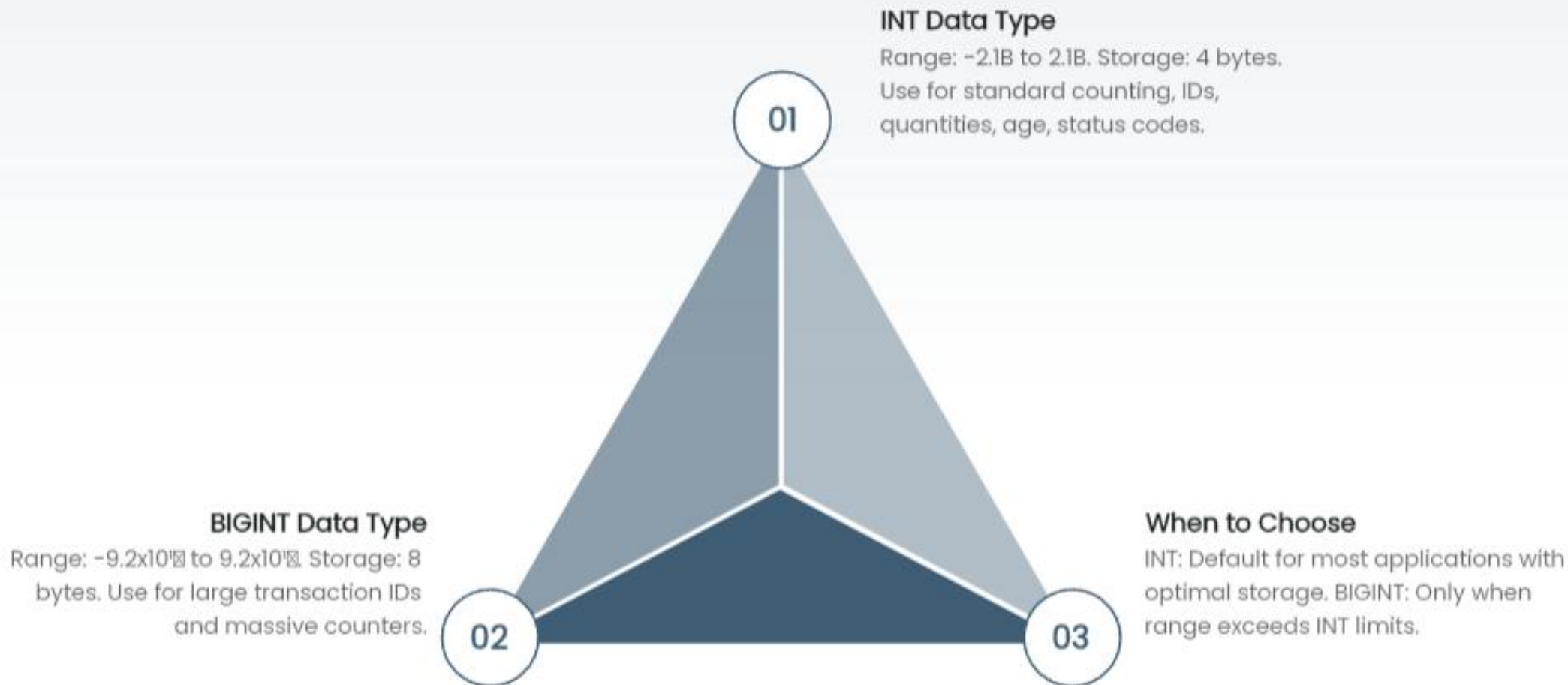
Factor	Without Proper Data Types	With Proper Data Types
Query Speed	Slower (Type Conversions)	Optimized Performance
Storage Cost	Higher (Wasted Space)	Reduced Footprint
Data Quality	Inconsistent	Enforced & Validated
Maintenance	Complex Debugging	Simplified Troubleshooting



02

Section 2: Core Data Type Categories

Exact Numerics: INT & BIGINT



Exact Numerics: DECIMAL & NUMERIC



DECIMAL(p,s) Format Definition

Precision (p) represents total digits from 1–38; Scale (s) represents decimal digits, ranging from 0–p maximum value.



Storage & Precision Example

DECIMAL(10,2) stores values up to 99,999,999.99 using 5–17 bytes depending on precision level requirements.



Performance Considerations

Slightly slower than INT operations but maintains accuracy; essential for precise financial calculations and data integrity.



Core Characteristics

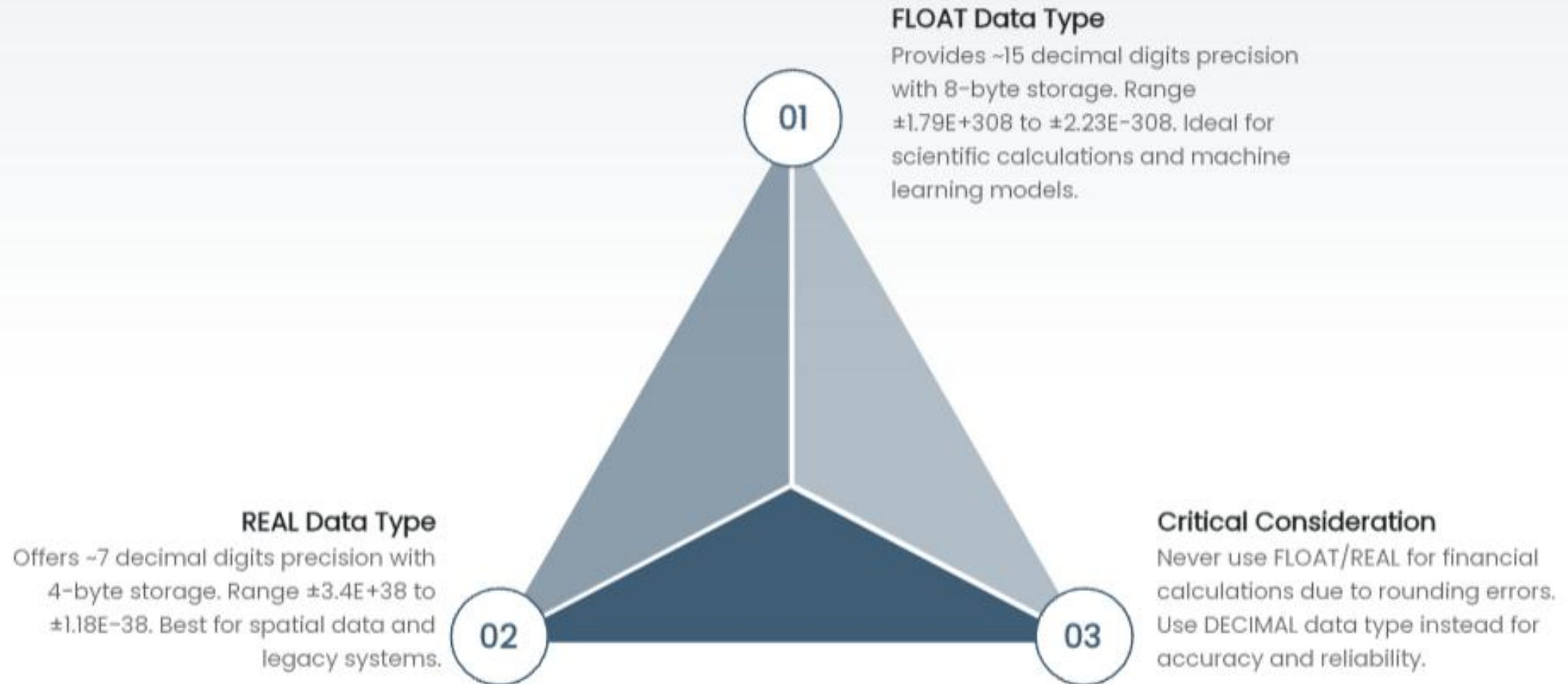
Fixed-point arithmetic eliminates rounding errors; ideal for financial data including currency, percentages, and interest rates.



Real-World Applications

Bank account balance uses DECIMAL(15,2); employee salary stored as DECIMAL(12,2) for accurate monetary representation.

Approximate Numerics: FLOAT & REAL



Date & Time Types



DATE Data Type

Stores date only (YYYY-MM-DD) in 3 bytes. Range spans 0001-01-01 to 9999-12-31. Ideal for birth dates and deadlines.



TIME Data Type

Stores time only (HH:MM:SS.nnnnnnn) in 3–5 bytes. Perfect for shift times and appointment scheduling requirements.



DATETIME Data Type

Combines date and time (YYYY-MM-DD HH:MM:SS.nnn) in 8 bytes. Suitable for transaction timestamps and order creation logs.



DATETIME2 Data Type

Enhanced version with up to 7-digit fractional seconds in 6–8 bytes. Ideal for high-precision event tracking and audit logs.



DATETIMEOFFSET Data Type

Includes timezone offset information in 10 bytes. Best for global applications and international scheduling needs.

Character String Types



CHAR(n) - Fixed-Length Storage

Fixed-length with space padding. Ideal for country codes, gender indicators, status codes. Stores 1–8000 bytes consistently.



VARCHAR(n) - Variable-Length Storage

Variable-length without padding. Perfect for names, addresses, emails. Efficient storage using only needed space for data.



VARCHAR(MAX) - Extended Storage

Variable-length up to 2GB capacity. Suitable for articles, JSON data, XML content, and large file content storage.



TEXT - Legacy Type

Deprecated data type; avoid in new projects. Use VARCHAR(MAX) instead for better performance and compatibility.

Unicode Character Strings

NCHAR(n)

Fixed-length Unicode type storing 1–8000 characters, 2 bytes per character for multilingual fixed data applications.

NVARCHAR(n)

Variable-length Unicode supporting unlimited storage with MAX option, ideal for global applications and emoji support.

Use Cases

Multilingual fixed data, country names in multiple languages, and international customer name storage requirements.

Storage Characteristics

Each Unicode character requires 2 bytes, doubling standard CHAR storage for comprehensive character set support.

Global Application Support

Supports 65,000+ characters, essential for Chinese, Arabic, Hindi text, and globalized application development needs.

NTEXT Type

Legacy Unicode type now deprecated; migrate existing implementations to NVARCHAR(MAX) for modern SQL Server compatibility.

Binary Data Types

01 FIRST

BINARY(n): Fixed-Length Storage

Fixed-length binary data storing 1–8000 bytes. Ideal for file headers, encryption keys, checksums, image fingerprints with consistent size requirements.

02 SECOND

VARBINARY(n): Variable-Length Storage

Variable-length binary up to 8000 bytes. Perfect for images, PDFs, encrypted passwords, video thumbnails with efficient storage and no padding waste.

03 THIRD

Encoding Requirements

Binary data needs BASE64 or HEX encoding for text representation, enabling proper data transmission and display in text-based systems and applications.

04 FOURTH

VARBINARY(MAX): Large-Scale Storage

Massive binary storage up to 2GB capacity. Handles entire document files, video clips, large encrypted data requiring substantial space allocation.

Other Specialized Data Types

BIT Data Type

Stores 0 or 1 values with efficient 1 byte per 8 bits storage for flags and boolean status indicators.

UNIQUEIDENTIFIER Data Type

128-bit unique identifier requiring 16 bytes, ideal for distributed systems and multi-database synchronization.

XML Data Type

Stores structured XML documents up to 2GB for configuration data, hierarchical records, and web service responses.

JSON Data Type

Stored as text with JSON validation, perfect for API responses and modern document-oriented applications.

BIT Use Cases

Active/Inactive flags, boolean status indicators, and feature toggles in database applications.

UNIQUEIDENTIFIER Use Cases

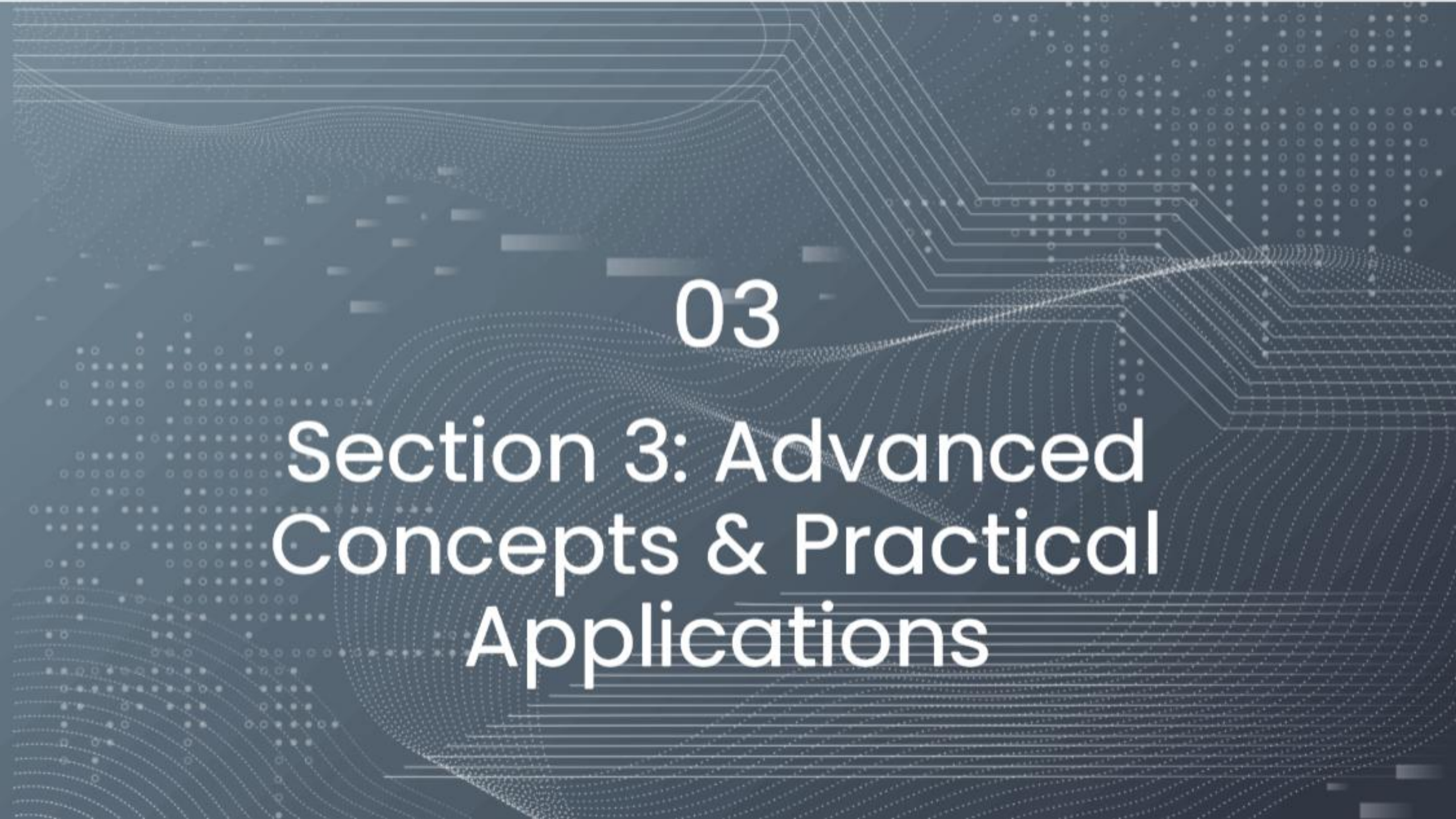
Distributed system primary keys and multi-database synchronization across connected systems.

XML Use Cases

Configuration data storage, hierarchical record management, and web service response handling.

JSON Use Cases

API response integration, document-oriented data storage, and modern application data management.



03

Section 3: Advanced Concepts & Practical Applications

Data Type Precedence & Conversion



Precision, Scale & Length Explained

Precision Definition

Total number of digits in a number. `DECIMAL(10,2)` has precision of ten, including digits before and after decimal point.

Precision Example

Product Price uses `DECIMAL(10,2)` storing up to \$9,999,999.99 maximum value for retail pricing applications.

Scale Definition

Number of digits after the decimal point. `DECIMAL(10,2)` has scale of two, must not exceed precision value.

Scale Example

Employee Salary uses `DECIMAL(12,2)` storing \$999,999,999.99. Discount Rate uses `DECIMAL(5,3)` for 99.999%.

Length Definition

Maximum character count in string types. `VARCHAR(50)` stores fifty characters; `NVARCHAR(MAX)` supports up to two gigabytes.

Length Application

Choose `VARCHAR` for fixed-size strings with character limits or `NVARCHAR(MAX)` for unlimited storage needs in text fields.



String Collation & Comparison



Collation Rules & Sensitivity

Collation governs string comparison, sorting, and storage. Case sensitivity uses CS_AS versus CI for case-insensitive modes.



Case Sensitivity Impact

SQL_Latin1_General_CI_AS enables case sensitivity. WHERE Name = 'john' matches 'JOHN' only in case-insensitive collation.



Server Default Behavior

SQL Server defaults to case-insensitive, accent-sensitive collation. Understanding defaults prevents unexpected query results.



Accent & Width Sensitivity

AS_KS_WS options control accent, Kana, and width sensitivity. These affect international character comparisons and sorting accuracy.



International Application Best Practice

Explicitly specify collation in international applications to ensure consistent comparison behavior across diverse character sets and languages.

Large Data Type Best Practices

VARCHAR(MAX) / NVARCHAR(MAX)

Use for variable-length text exceeding 8000 characters. Store separately with foreign keys for very large objects to optimize performance.

VARBINARY(MAX)

Ideal for binary files like images and PDFs. Evaluate external blob storage for very large files to balance performance and costs.

Performance Considerations

VARCHAR(n) outperforms MAX types for smaller data. Choose based on actual data size and access patterns for optimal efficiency.

Storage Strategy Guidelines

Small attachments under 100MB use VARBINARY(MAX). Large attachments over 100MB reference external storage with database URLs instead.

External Storage Benefits

Azure Blob and AWS S3 reduce database overhead. Improves scalability, reduces costs, and enhances overall application performance significantly.

Trade-off Analysis

Balance database storage capacity against application performance requirements and infrastructure costs for optimal solution selection.

Implementation Recommendation

Evaluate data size distribution first. Design hybrid approach combining database and external storage for cost-effective scalability.

Monitoring Best Practices

Track storage usage continuously. Audit access patterns regularly to optimize strategy and adjust storage allocation accordingly.

User-Defined Data Types (UDT)

Alias Types Overview

Wrapper around built-in types with custom names and default values for consistent definitions across tables and improved readability.

01

.NET CLR Types Overview

Custom types created from .NET languages for advanced scenarios like geospatial types and complex business object representations.

02

UDT Use Cases

Standardizing common data representations, enforcing domain rules, and improving data consistency across database applications.

03

Alias Types Benefits

Enforces consistent definition across tables, improves code readability, and standardizes common data representations effectively.

04

.NET CLR Requirements

Requires CLR enabled on SQL Server instance to implement custom types and enforce domain rules effectively.

05

UDT Limitations

User-defined types are aliases; actual storage uses underlying base type, not separate storage mechanism.

06

INT vs BIGINT: Storage & Performance

Storage & Range Characteristics

INT uses 4 bytes with range -2,147,483,648 to 2,147,483,647; BIGINT uses 8 bytes supporting significantly larger values for enterprise-scale applications requiring extended numeric capacity.

Performance & Practical Applications

INT offers faster query performance with smaller indexes, ideal for user IDs and quantities; BIGINT accommodates distributed systems and millisecond timestamps but requires larger storage overhead.



Aspect	INT	BIGINT
Storage	4 bytes	8 bytes
Max Value	2,147,483,647	9,223,372,036,854,775,807
Min Value	-2,147,483,648	-9,223,372,036,854,775,808
Performance	Faster (smaller index)	Slower (larger index)
Query Joins	Efficient	Acceptable
Default Choice	Recommended	Use only if needed
Example Use	User IDs, Quantities	Distributed IDs, Millisecond timestamps



DECIMAL vs FLOAT Comparison

Key Differences

DECIMAL offers fixed-point precision ideal for financial data, while FLOAT provides approximate floating-point calculations optimized for scientific computing and performance-critical applications.

Practical Applications

Use DECIMAL for currency and accounting requiring exact values; employ FLOAT for physics simulations, machine learning models, and scenarios where hardware acceleration benefits outweigh precision requirements.

Aspect	DECIMAL	FLOAT
Type	Fixed-point	Approximate (floating-point)
Accuracy	Exact	Rounding errors possible
Storage	5–17 bytes	8 bytes
Calculations	No loss	Precision loss over iterations
Financial Data	✓ Perfect	✗ Avoid
Scientific Data	Acceptable	✓ Ideal
Performance	Slightly slower	Faster (hardware-accelerated)
Example Use	Currency, accounting	Physics, ML models

CHAR vs VARCHAR Comparison

Storage and Performance

CHAR uses fixed storage (always n bytes with padding), while VARCHAR uses variable storage (only used bytes). CHAR retrieves marginally faster; VARCHAR wastes no space but retrieves slightly

Use Cases and Examples

CHAR suits fixed-length data like country codes and gender values. VARCHAR suits variable-length data like names and descriptions. VARCHAR is generally recommended as the default choice for

Storage Efficiency

CHAR(10) storing 'USA' uses 7 bytes with padding; VARCHAR(10) storing 'USA' uses only 3 bytes. High wastage occurs with CHAR when average length significantly less than n.

Selection Recommendation

Use CHAR only for truly fixed-length data requiring marginal speed gains. VARCHAR is the default choice for most scenarios, offering better space efficiency and flexibility.

Aspect	CHAR(n)	VARCHAR(n)
Storage Type	Fixed	Variable
Space Usage	Always n bytes (padded)	Only used bytes
Retrieval Speed	Marginally faster	Marginally slower
Ideal For	Fixed-length data	Variable-length data
Wastage	High if avg length << n	None
Example	Country codes, Gender	Names, Descriptions
Storage Example	CHAR(10) 'USA' = 7 bytes	VARCHAR(10) 'USA' = 3 bytes
Recommendation	Use only for fixed data	Default choice

VARCHAR vs NVARCHAR Comparison

Storage and Character Support

VARCHAR uses 1 byte per ASCII character for efficient storage;
 NVARCHAR uses 2 bytes per Unicode character, supporting 65,000+
 multilingual characters including emoji and international scripts.

Practical Application Scenarios

VARCHAR suits legacy systems and ASCII-only data like email
 addresses; NVARCHAR is essential for modern global applications
 requiring international customer names, addresses, and multilingual
 content support.

Aspect	VARCHAR(n)	NVARCHAR(n)
Character Set	ASCII (Latin)	Unicode (multilingual)
Bytes Per Char	1 byte	2 bytes
Storage Efficiency	✓ Compact	Uses 2x space
International Text	Limited (A-Z, 0-9)	✓ Full support (Chinese, Arabic, emoji)
Language Support	English, Latin	65,000+ characters
Global Apps	✗ Not suitable	✓ Essential
Example	Email (mostly ASCII)	Customer names (international users)
Recommendation	Legacy/ASCII-only	Modern/Global applications

Real-World Use Case: E-Commerce System

Product Catalog Table Structure

Database table stores product information using VARCHAR for names, INT for inventory counts, and DECIMAL for pricing data accurately.

Data Type Selection Rationale

VARCHAR ensures flexible text storage, INT handles quantity tracking efficiently, DECIMAL maintains precise financial calculations for transactions.

Performance Optimization Benefits

Proper data type assignment reduces storage overhead, accelerates query execution, and ensures accurate reporting for inventory management systems.

Practical Implementation Example

Product ID uses INT, name uses VARCHAR(255), price uses DECIMAL(10,2), stock quantity uses INT for streamlined e-commerce operations.



The background features a dark blue-grey color with a light grey hexagonal grid pattern. Several icons are placed within the grid: a laptop icon in the top left, a cloud icon below it, a globe icon in the middle right, a gear icon in the bottom right, and a share icon in the bottom right corner.

Thanks